

Topics in Software Engineering – Software Reengineering

By

Dr. Junaid Akram

Assistant Professor, Department of Computer Science COMSATS (Lahore)

Legacy Systems

Concepts and Methods

Outline

- Introduction
- Legacy System Structures
- Legacy System Design
- Legacy System Assessment

Introduction..

Legacy systems: old computer-based systems still in use by organizations

- Many of them still business critical
- Incorporate many changes made over the years
- Many people have been involved in these changes

Replacing legacy systems with new systems is risky, yet keeping them means new changes become more and more expensive

Risks of Replacing a Legacy System

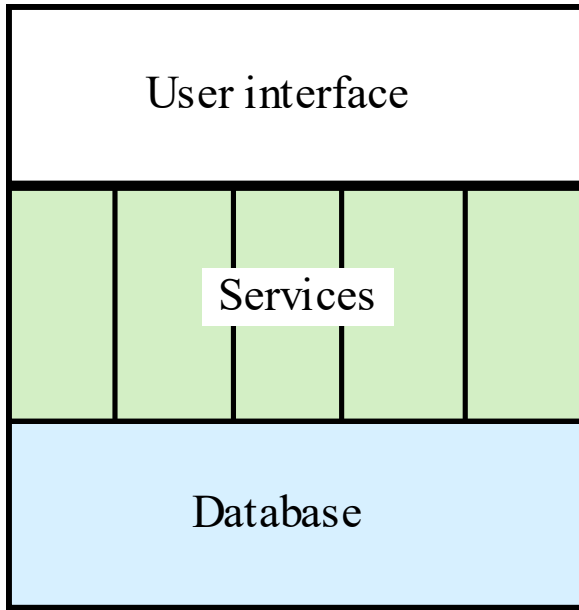
- Specification is difficult because existing documentation is typically **incomplete**
- Changing business processes may entail **high costs**
- **Undocumented**, yet important business rules may be embedded in the system; a new system may break these rules

**“The new system may be delivered late, may cost more than expected,
and may not function properly”**

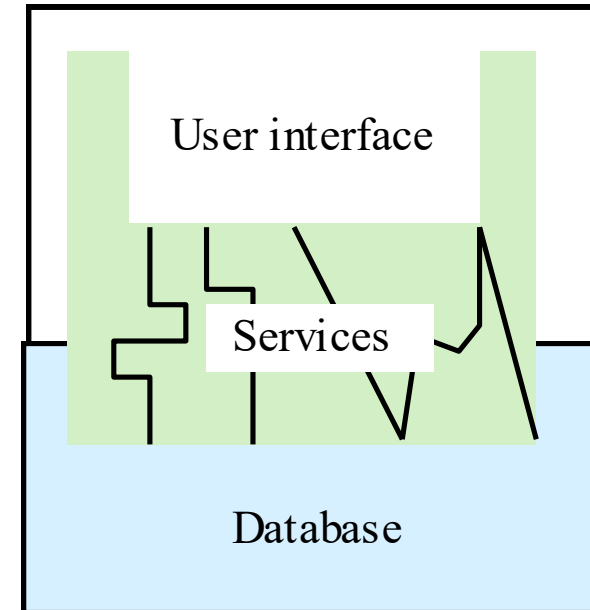
Factors that make changes to legacy systems expensive:

- In large systems, **different parts** were implemented by different teams, without consistent **programming style**
- It is difficult to find person who knows the **obsolete programming languages** used in old systems
- In many cases the only documentation is provided to change
- Data used by the system is difficult to understand and manipulate; it can also be **obsolete** and/or **redundant**

Legacy system structures



Ideal model for distribution



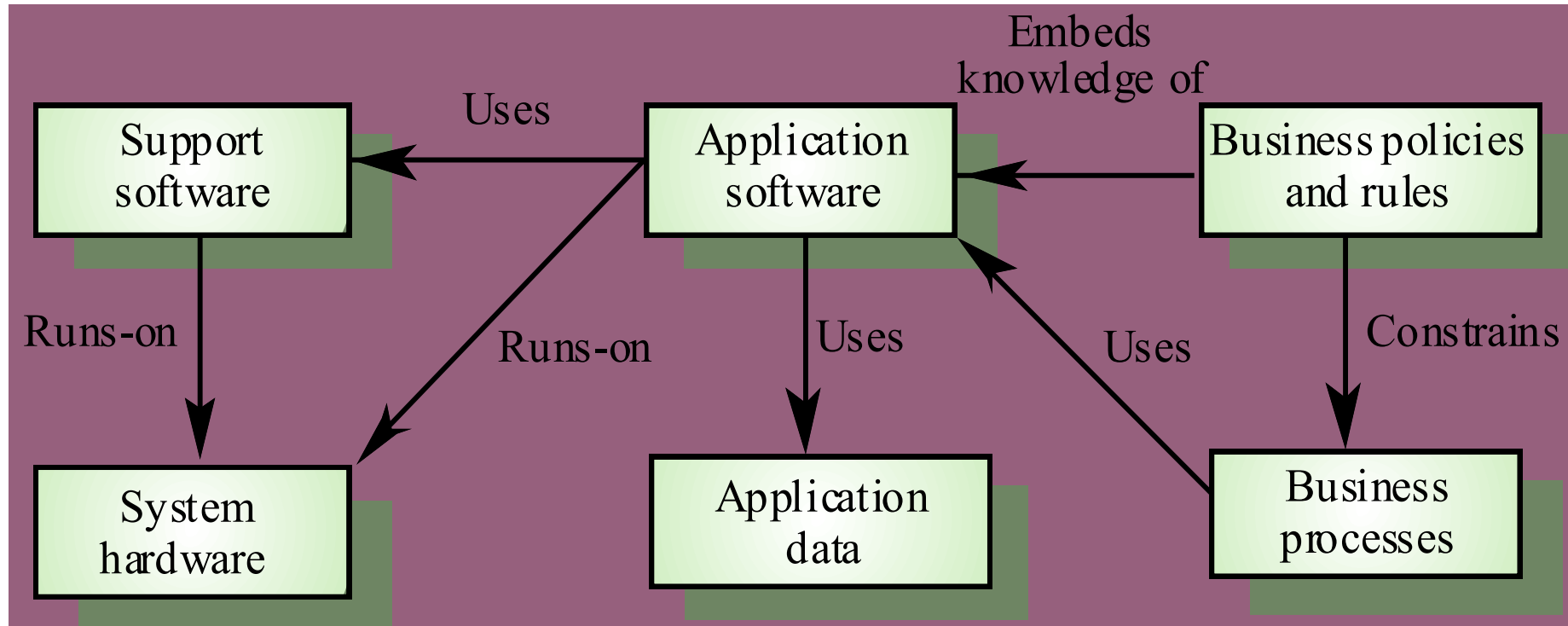
Real legacy systems

Legacy system structures....

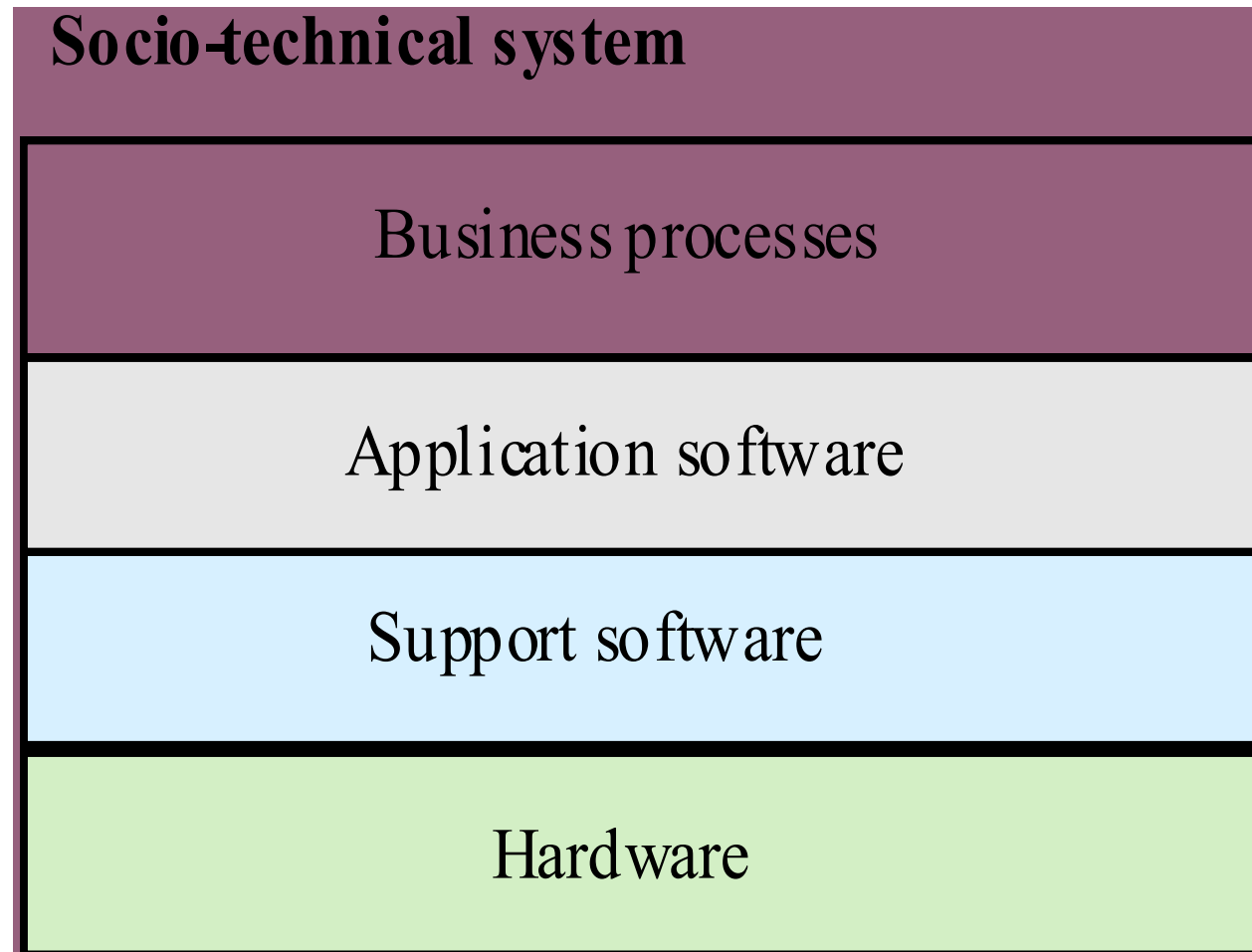
Legacy systems involve more than software (they are computer-based systems). Typical logical parts of a legacy system are:

- System **hardware**
- Support **software**
- Application software (legacy software systems)
- Application **data**
- Business processes
- Business **policies and rules**

..Legacy system components

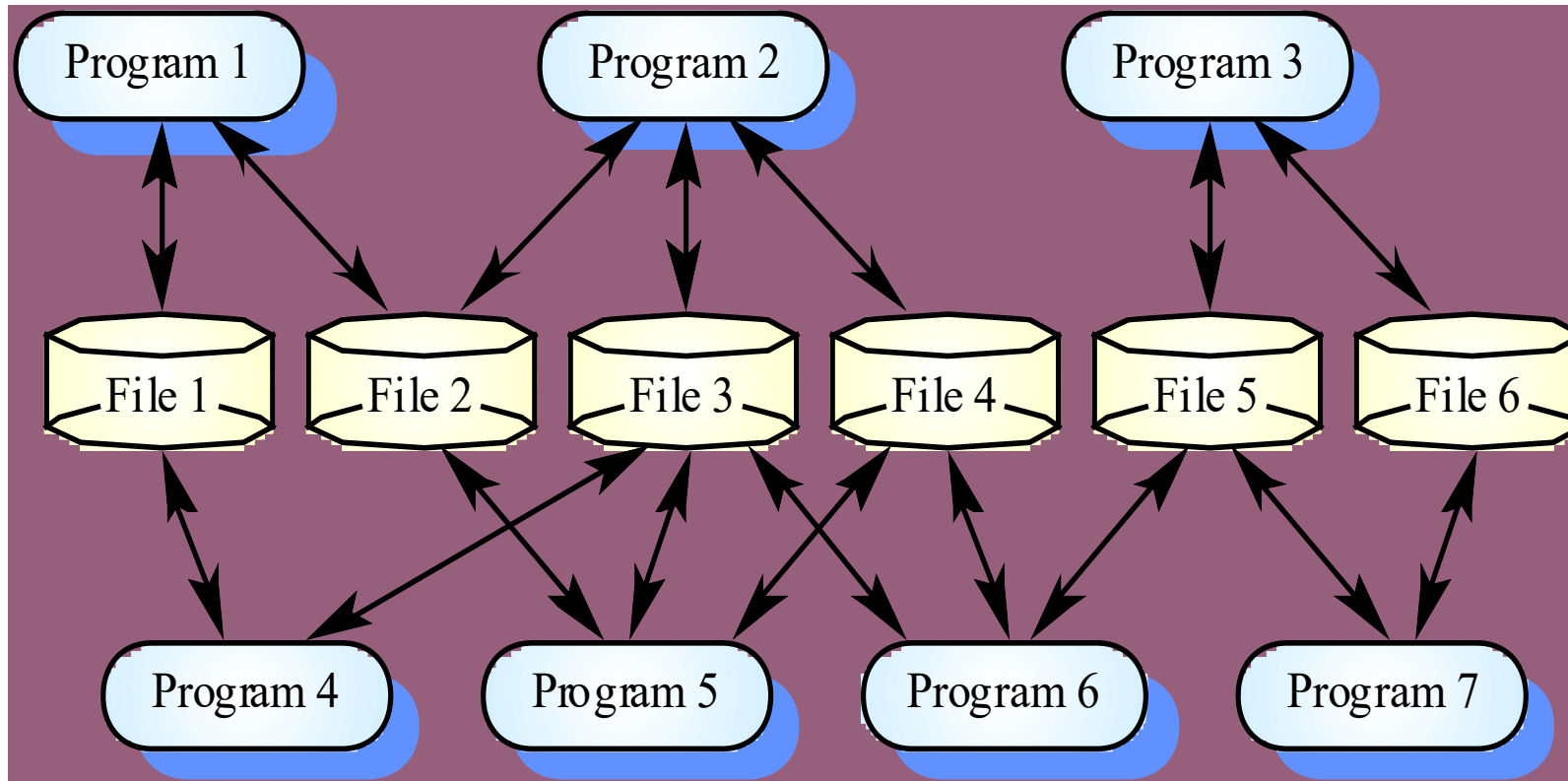


..Alternative view of legacy systems



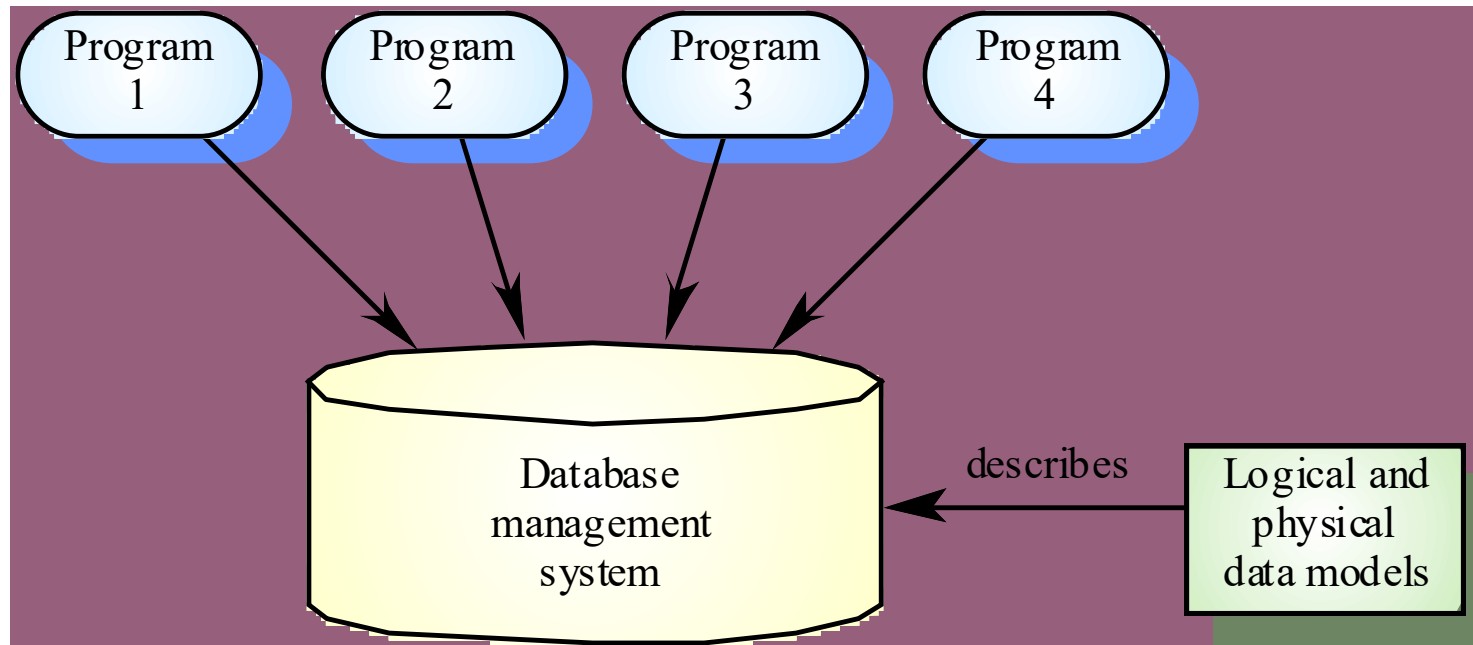
..Legacy application

- Both programs and data files have been added over the years: components are heterogeneous and strongly coupled



..Database-centered legacy systems

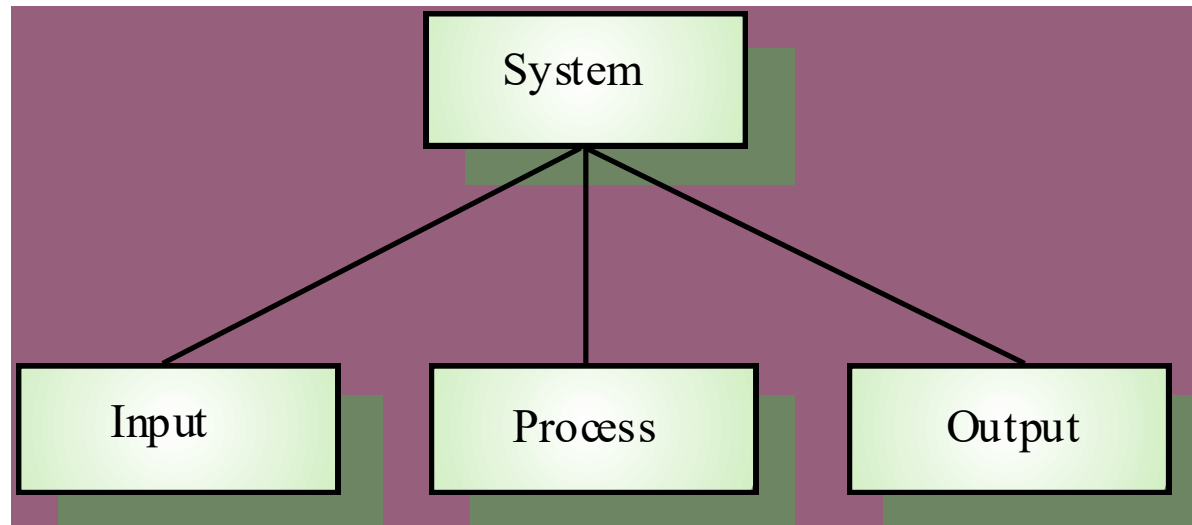
- **Advantages:** logical and physical data models are used, redundancy is reduced, impact of system changes can be easier assessed, recovery is possible
- **Main issues:** DBMS may be obsolete, accessing software (“teleprocessing monitor”) may need to be replaced



Legacy system design...

- Legacy systems design is typically function oriented. Two main classes:
 - *Batch processing systems*: both input and output is provided in “batches,” e.g., a payroll system
 - *Transaction processing systems*: input & output related to a database transaction, e.g., a flight reservation system
- Both batch processing and transaction processing systems usually follow an *IPO model*

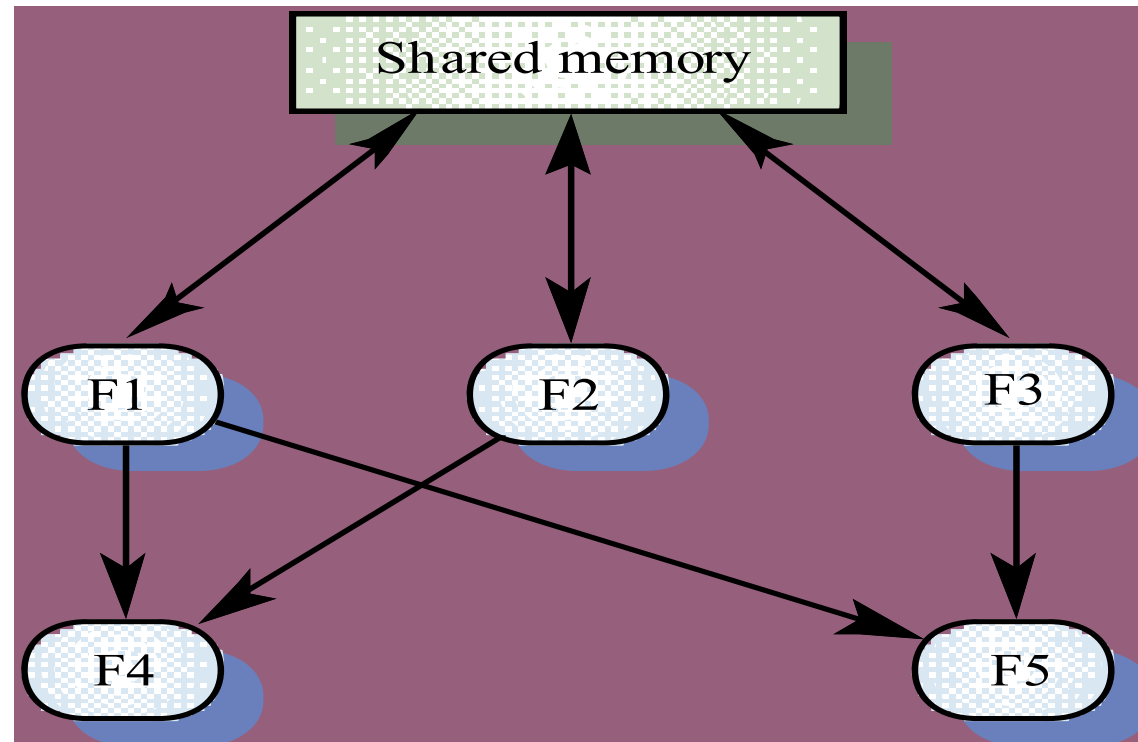
The IPO Model



- **I**nput: inputs are collected from one or more sources
- **P**rocessing: some computations are performed on inputs
- **O**utput: results are provided either in batches or as single-transaction outputs

A function-oriented view on design

The main problem is the **sharing of data**: It is unlikely that a single person understands all parts of a large system.



Legacy system assessment.....

Strategic approaches for dealing with legacy systems:

- Scrap the system completely
 - When business practices have changed and no longer depend significantly on the system
- Continue to maintain the system
 - When the system works well, is fairly stable, and users do not request many changes
- Transform the system to improve maintainability
 - When system quality was affected negatively by changes, yet changes are still required
- Replace the system with a new one
 - When obsolete hardware precludes further operation or the new system can be built at reasonable cost

Legacy system assessment...

❑ Business value assessment (subjective). Viewpoints:

- **End-users:** look at system's functionality and performance
- **Customers:** look at the quality of services provided
- **Business managers:** assess the usefulness of the system in terms of business support
- **IT managers:** are concerned with the availability of technical support for the system
- **Senior managers:** interested in the system's contribution to the business goals

Legacy system assessment.. (System Quality Assessment)

❑ Business process assessment. Possible questions:

- Are defined process models and procedures in place?
- Are processes applied consistently across the company?
- What adaptations have been made?
- Are relationships with other business processes necessary?
- Are processes suitably supported by application software?

❑ Environment assessment: support software & hardware platform (maintenance costs, faults, etc)

❑ Application software assessment factors

- Number of system **change requests**
- Number of different **user interfaces**
- **Volume of data** used by the system

Legacy System Assessment (Factors in environment assessment)

Factor	Questions
Supplier stability	Is the supplier is still in existence? Is the supplier financially stable and likely to continue in existence? If the supplier is no longer in business, are the systems maintained by someone else?
Failure rate	Does the hardware have a high rate of reported failures? Does the support software crash and force system restarts?
Age	How old is the hardware and software? The older the hardware and support software, the more obsolete it will be. It may still function correctly but there could be significant economic and business benefits to moving to more modern systems.
Performance	Is the performance of the system adequate? Do performance problems have a significant effect on system users?
Support requirements	What local support is required by the hardware and software? If there are high costs associated with this support, it may be worth considering system replacement.
Maintenance costs	What are the costs of hardware maintenance and support software licences? Older hardware may have higher maintenance costs than modern systems. Support software may have high annual licensing costs.
Interoperability	Are there problems interfacing the system to other systems? Can compilers etc. be used with current versions of the operating system? Is hardware emulation required?

Cont..

Factor	Questions
Understandability	How difficult is it to understand the source code of the current system? How complex are the control structures which are used? Do variables have meaningful names that reflect their function?
Documentation	What system documentation is available? Is the documentation complete, consistent and up -to-date?
Data	Is there an explicit data model for the system? To what extent is data duplicated in different files? Is the data used by the system up-to-date and consistent?
Performance	Is the performance of the application adequate? Do performance problems have a significant effect on system users?
Programming language	Are modern compilers available for the programming language used to develop the system? Is the programming language still used for new system development?
Configuration management	Are all versions of all parts of the system managed by a configuration management system? Is there an explicit description of the versions of components that are used in the current system?
Test data	Does test data for the system exist? Is there a record of regression tests carried out when new features have been added to the system?
Personnel skills	Are there people available who have the skills to maintain the application? Are there only a limited number of people who understand the system?

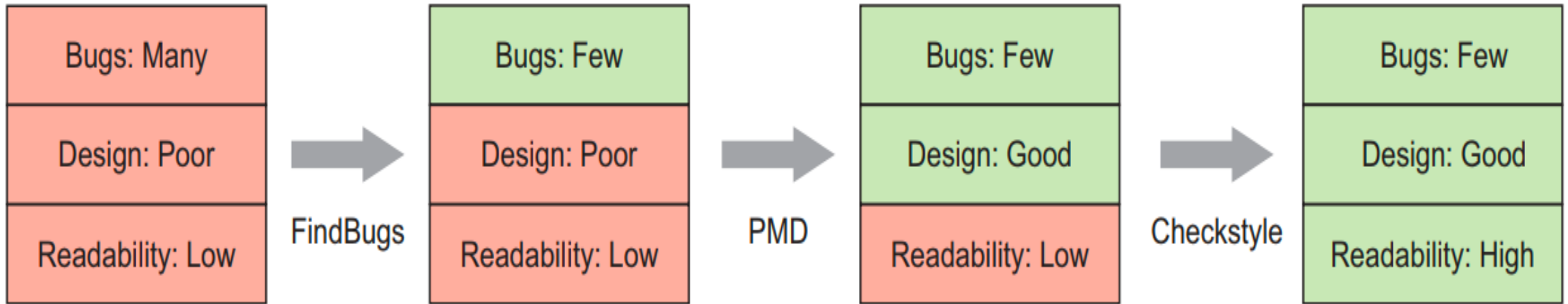
Working Effectively with Legacy Systems.

A thick, hand-drawn style orange line that underlines the title text.

Changes, Benefits and Risks for a Legacy Project

Changes	Benefits	Risks	Actions required
Removing an old feature	• Easier development • Better performance	• Somebody is still using the feature	• Check access logs • Ask users
Refactoring	• Easier development	• Accidental regression	• Code review • Testing
Upgrading a library	• Bug fixes • Performance improvements	• Regression due to a change in the library's behavior	• Read the changelog • Review library code • Manually test major features

Improving code using static analysis tools



An example FindBugs analysis result

FindBugs-IDEA FindBugs Analysis Results

- ▼ camel (found 210 bug items in 1634 classes) [more...](#)
 - ▶ **Bad practice (66 items)**
 - ▶ Performance (14 items)
 - ▶ Multithreaded correctness (28 items)
 - ▶ Experimental (1 item)
 - ▶ Internationalization (23 items)
 - ▶ Malicious code vulnerability (45 items)
 - ▼ Dodgy code (23 items)
 - ▼ RuntimeException capture (1 item)
 - ▶ Exception is caught when Exception is not thrown (1 item)
 - ▼ Bad casts of object references (3 items)
 - ▶ Unchecked/unconfirmed cast (2 items)
 - ▶ instanceof will always return true (1 item)
 - ▼ Null pointer dereference (4 items)
 - ▶ Load of known null value (4 items)
 - ▼ Redundant comparison to null (8 items)
 - ▶ Redundant nullcheck of value known to be non-null (8 items)
 - ▼ Unread field (6 items)
 - ▶ Unread public/protected field (6 items)
 - ▼ Unused field (1 item)
 - ▶ Unused public or protected field (1 item)
 - ▼ Correctness (10 items)
 - ▼ Null pointer dereference (3 items)
 - ▶ Non-virtual method call passes null for nonnull parameter (1 item)
 - ▶ Possible null pointer dereference (1 item)
 - ▶ Method call passes null for nonnull parameter (1 item)
 - ▼ Dubious method used (1 item)
 - ▶ Futile attempt to change max pool size of ScheduledThreadPoolExecutor (1 item)
 - ▼ Redundant comparison to null (2 items)
 - ▶ Nullcheck of value previously dereferenced (2 items)
 - ▼ Bad use of return value from method (2 items)
 - ▶ Method ignores return value (2 items)
 - ▼ Masked Field (2 items)
 - ▶ Class defines field that masks a superclass field (2 items)

Stop writing legacy code!

Foster communication

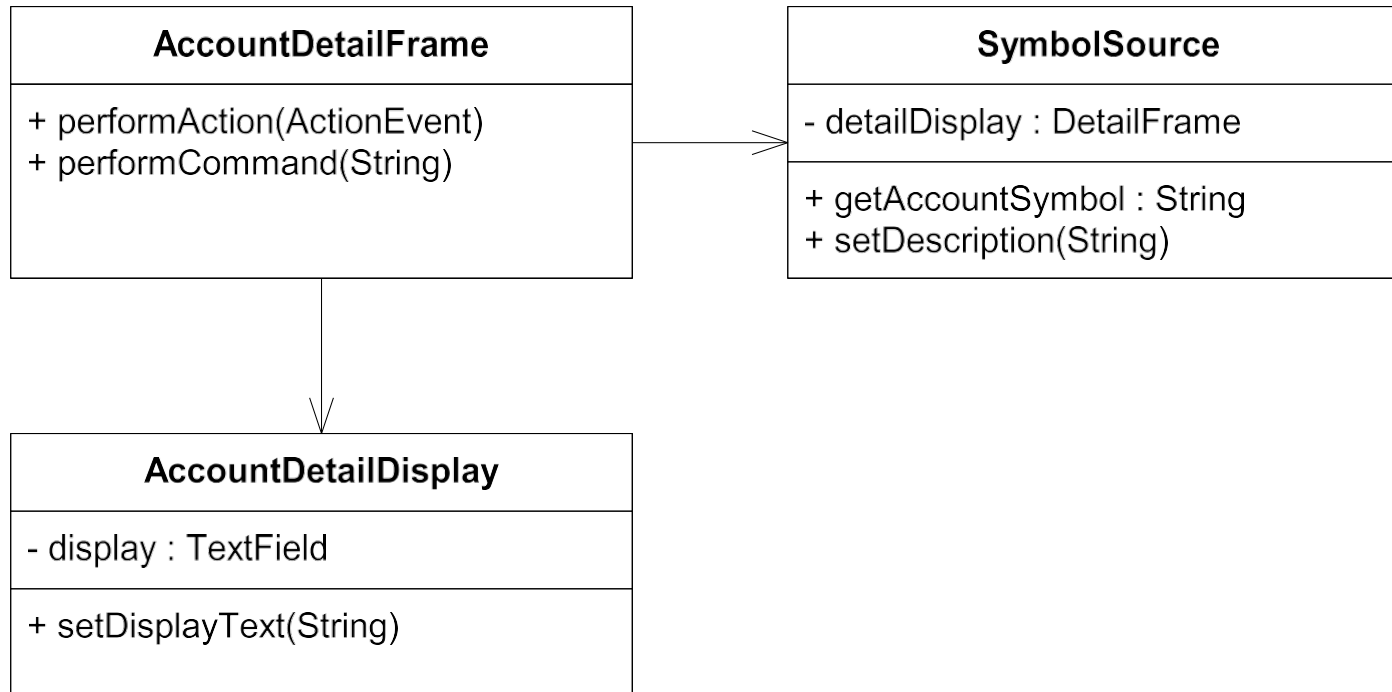
- **Code reviews**— All changes to the code should be reviewed by at least one other developer. It's worth taking the time to find a code review system that works well for your team.
- **Pair programming**— The ideal to strive for is a situation whereby developers start pairing without being asked to.
- **Tech talks**— Give the presenter a chance to show off their skills or something they've built, and everybody else gets to learn about what other people are up to.
- **Present your projects to other teams**— Setting up a semi-regular session where each development team presents a technical overview of one of their products (legacy or otherwise) can be really valuable.
- **Hack days**— This is a chance for developers to work with people from different teams, play with new technologies, and build cool stuff, preferably outside of the office environment.

Periodic Code Reviews

- Ask everybody to take an hour or so to **look through the code** beforehand and take notes. This means people can spend the review discussing things, rather than just staring at the code in silence.
- Ask one person who is knowledgeable about the code to **lead the review**. They'll spend a few minutes introducing the codebase and then go around the room asking people for their comments.
- Write up a list of **the review's conclusions**, divided into concrete actions and non-specific ideas or things to investigate.
- **Share the review's document** with the team and ask them to update it when they complete any of the actions. Check on progress a few weeks later.

Making it better

We can make the design better by extracting classes for those separate responsibilities



Strategies for breaking dependencies

Lean on tools

- Use tools with automated refactorings

- Mock libraries

If manual, then

- Lean on the compiler

 - Use the compiler to report points of change by deliberately producing errors

- Preserve signatures

 - Favor refactorings which allow you to cut/copy and paste signatures without modification

- Single goal editing

 - Do one thing at a time!

Irritating parameters

```
public class CreditValidator {
```

```
    public CreditValidator(RGHConnection connection,  
        CreditMaster master,  
        String validatorID) {  
        ...  
    }
```

How am I
going to
construct these
parameters
for the test?

Setting up network connection is
not possible...

```
    Certificate validateCustomer(Customer customer)  
        throws InvalidCredit {  
        ...  
    }  
    ...  
}
```

Hidden dependency

Consider this C++ constructor:

```
mailing_list_dispatcher::mailing_list_dispatcher()  
    : service(new mail_service), status(MAIL_OKAY)  
{  
    const int client_type = 12;  
    service->connect();  
    status = MAIL_OFFLINE;  
    ...  
}
```

This constructor
relies on the class
mail_service in the
initialization list

We don't want to initialize the mail_service, because then we connect to the network and start sending actual mails...

Hidden dependency: Solution

Parameterize constructor

```
mailing_list_dispatcher::mailing_list_dispatcher  
    (mail_service *service) : status(MAIL_OKAY)
```

Big improvement? (Yes)

- Allows for introducing a fake mail service

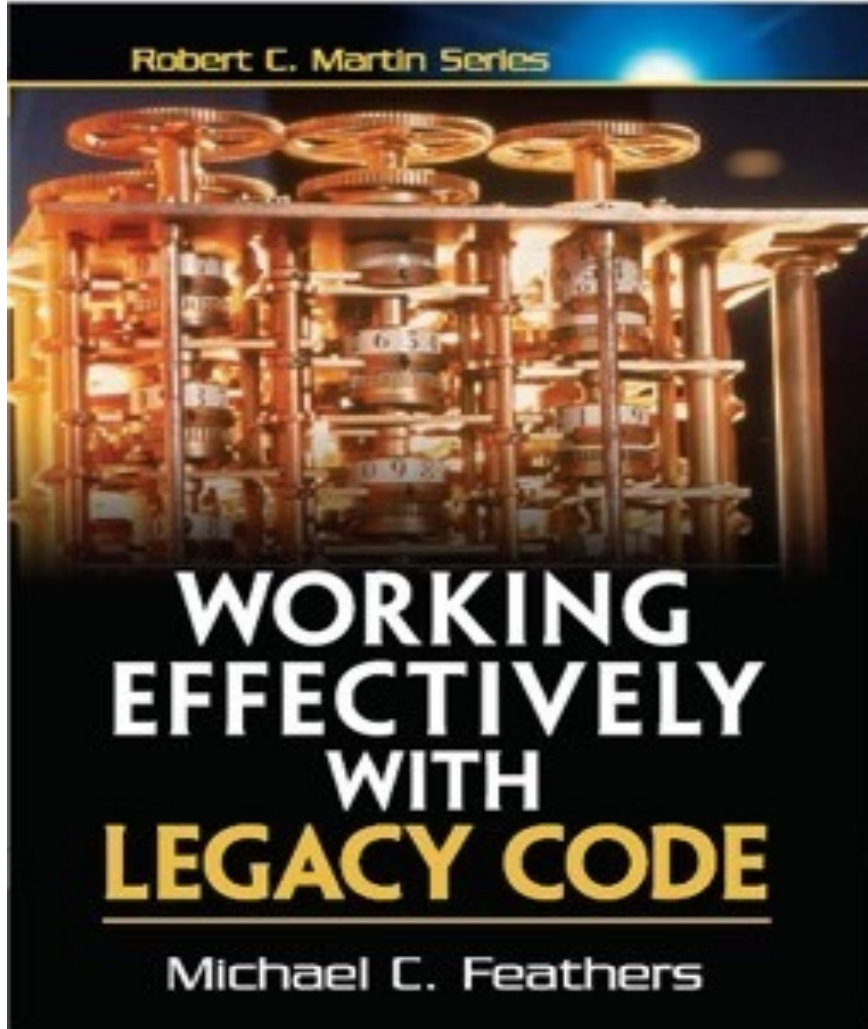
 - Extract interface for mail_service

 - Introduce fake class that senses the things we do

Summary

- If you want to keep your project healthy, don't focus only on the source code. Documentation, toolchain, infrastructure, automation, and the culture of the team are all important.
- Information about your software will gradually leak away, unless you constantly guard against it.
- Good technical documentation is worth its weight in gold. And a team that communicates so well it doesn't need documentation is even better.
- You need documentation to prevent knowledge being lost as team members leave.
- Build large software out of components small enough to be discarded and rewritten without risk.
- Automate every process and procedure

More info and strategies



Working Effectively with Legacy Code

Michael Feathers, Prentice Hall, 1 edition, 2004

Thanks for your attention!

Any Question?



Email me on : junaidakram@cuilahore.edu.pk